PA2: Knight's Treasure Hunt

Andy, Yu-Guang Chen
Associate Professor, Department of EE
National Central University
andyygchen@ee.ncu.edu.tw
Slides Credit: TA Tse-Chuan Yang



Problem

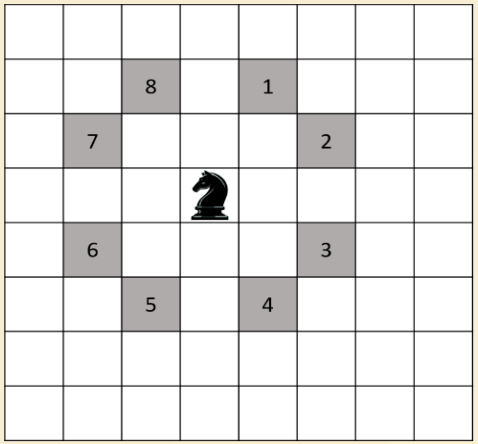
This assignment requires you to complete a game based on the Knight's Tour problem.

- **Background:** The Knight's Tour is a classic chess puzzle where a knight moves in an L-shape and tries to visit every square once.
- **Modified Task:** You must find the highest-scoring path under step and energy constraints.
- **Goal:** Use **recursion with pruning** to search for the best path from the starting point.
- **Output:** Display the final score, replay steps, and the best path grid showing the knight's route.



Movement Rules

- The knight moves in a standard **L-shape**, each valid move gives **+1 score** and consumes **1 energy.**
- It **cannot revisit** any square or exceed step limit,and stops when **energy runs out.**
- The goal is to find the **best-scoring path**, not necessarily to exhaust all energy or steps, and the result is **not unique.**



Board Tokens Type

S: Starting point ♞

B: Bonus — adds extra score. 🪙

E: Energy — restores energy. ⚡

X: TNT — reduces score. 💣

#: Guard Ray — impassable obstacle. 🚫

O: Empty square — normal step. ☐



Input Format

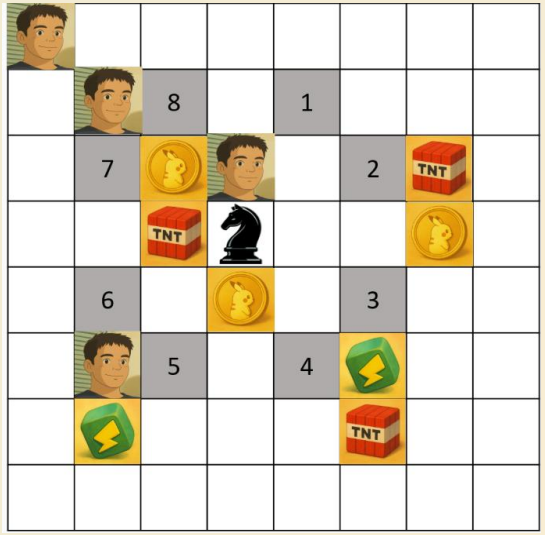
- You will need to enter **board size N**, **step limit L**, and **starting energy E** in the **first line**.
- Then enter **bonus score S**, **energy value V**, and **bomb penalty P** in the **second line**.


+ S score


+ V energy



- P score

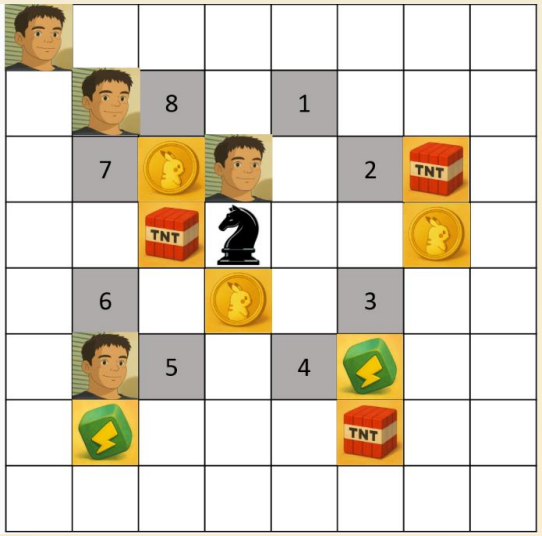


NxN board

Input Format

- Finally, enter the **board configuration** — an **N×N grid**, each line containing N tokens separated by spaces.
- Example of valid 8×8 board configuration

```
# O O O O O O O
O # O O O O O O
O O B # O O X O
O O X S O O B O
O O O B O O O O
O # O O O E O O
O E O O O X O O
O O O O O O O O
```



Hint

- Use **recursion (DFS)** to explore all potential routes.
- You need **pruning** to cut off unpromising paths early. Example: stop recursion if current score + remaining maximum potential < best score.
- You can use **heuristic weighting** to evaluate the next move, and Reference **Warnsdorff's rule** to prefer moves with fewer future options.



Output Format

- First, display result of Knight's Treasure Hunt.

```
Best score: 4
Steps: 3
```

(it must be unique.)

(Y legal steps is OK.)

- Then, show the replay of Knight's Treasure Hunt.

```
Step   (x,y)      Type          Score      Energy
-------------------------------------------------
0      (0,3)      start         0          4
1      (2,2)      bomb          0          3
2      (0,1)      normal        1          2
3      (2,0)      bonus         4          1
```

Print steps 0 ~ Y



Output Format

◆Finally, print the Best Path Grid.

➢ Keep the starting point **S**.

➢ Mark the knight's path from **1** to **Y** (number of steps).

➢ All **unvisited** squares remain **unchanged**.

S 0 0 0

0 X 1 #

2 X 0 0

0 0 3 0



Bonus Part

Who is Andy?

- **A**: Guard Andy is an **impassable obstacle** that will **expand over time**.



- He has **four** growth stages:
  1x1 → 3x3 → 5x5 → 7x7
- Each Andy grows simultaneously and **overwrites** the map area.

Bonus Part

- After each move, **apply Andy's growth first.** Knight's shouldn't stay **inside any Andy's zone.**

step	size	Event Message
0	1*1	<!> OH NO <!> ANDY HAS AWAKENED!!! The FINAL COUNTDOWN begins...
5	3*3	<!> WARNING <!> ANDY IS ENRAGED!!! Run if you can...
10	5*5	<!> DANGER <!> ANDY GROWS STRONGER!!! You feel the end is near...
15	7*7	<!> CRITICAL <!> ANDY UNLEASHES CHAOS!!! Farewell, hero. This will be the end of your journey...
20	7*7	(Andy is at maximum size — no new messages)



Bonus Part I/O Format

On the final grid:

- Unvisited squares under Andy → print **A**.
- Visited squares keep **S** or their **step numbers**.

Input

```
A O # O E O O O A
O O O O O O E E O
O B O O X O X O O
O O E B O O B # O
# X O X S O X O B
X X X O B X O B O
B X # O E O O O E
O O E B O O B # O
A X O X O X X O A
```

Output

```
A  A  A  A  E  A  A  A  A
A  A  A  2  O  A  A  A  A
A  3  A  A  X  A  A  A  A
A  A  1  4  17 A  11 A  A
#  X  16 X  S  5  X  O  10
A  A  A  A  15 A  9  12 A
A  A  A  A  6  13 A  A  8
A  A  A  14 O  A  7  A  A
A  A  A  A  O  A  A  A  A
```





Hint

- Use **pair** to store coordinates (x, y).
- Example:

  pair<int, int> pos = {2, 5};
- Use **sort()** from C++ STL to reorder moves or data.(ascending in default)
- Example:

  vector<pair<int,int>> v = {{2,5}, {1,7}, {2,3}, {1,9}};
  sort(v.begin(), v.end());









Report

- How to compile and execute your program; (You can use a screenshot to explain. In this PA, it can be done in one click)
- The degree of completion of the assignment; (If you complete all requirements, just specify all)
- The hardness of this assignment and how you overcome it;
- Bonus function(s) you implement if any;
- Any suggestions about this programming assignment?
- You can also put anything related to the PA in your report, such as pseudocode, control flow diagram, programming development thoughts, etc.





Correctness

- Your program (basic & bonus part) will be evaluated by our checker using both **public and hidden testcases**.
- The hidden testcases are usually more harder, so make sure your code can deal with **complex condition**.
- OS: Ubuntu 20.04 LTS
- Compiler：**GNU** GCC 9.3.0
- Supporting C++ version：**C++ 11**



Q&A

- If you have any other problems, you can also send an email to me.
- TA email is kevin200481211@gmail.com
- Please cc andyygchen@ee.ncu.edu.tw



Thank you!



